

Trabalho 4 - Algoritmos de Busca

Identificação do Aluno

- **Nome:** Rafael Lima Sant'Ana
- **Matrícula:** 251035659
- **Disciplina:** FGA0147 - Estruturas de Dados 1
- **Turma:** T04

Exercício Prático

Código-fonte (main.c)

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

typedef struct {
    int ibge;
    char *municipio;
    char *uf;
    char *regiao;
    long populacao;
    char *porte;
} cidade;

typedef struct {
    int ibge;
    int linha;
} indice;

const int QTD_CIDADES = 5570;

char* substring_by_index(char *src, int start, int end);
void print_municipio(cidade *c);
int busca_linear(indice *arr, int n, int required_ibge);
int busca_binaria(indice *arr, int n, int required_ibge);
void liberar_cidades(cidade *arr, int n);
int carregar_dados(cidade **out_cidades, indice **out_indices, int *out_n);
```

```

int main() {
    cidade *cidades = NULL;
    indice *indices = NULL;
    int n = 0;
    int opcao;

    do {
        printf("\n===== MENU =====\n");
        printf("1. Carregar dados\n");
        printf("2. Consultar municipio\n");
        printf("3. Sair\n");
        printf("Escolha uma opcao: ");

        if (scanf("%d", &opcao) != 1) {
            int c;
            while ((c = getchar()) != '\n' && c != EOF);
            printf("Entrada invalida.\n");
            continue;
        }

        switch (opcao) {
            case 1: {
                if (cidades != NULL) {
                    printf("Dados ja carregados. Recarregando...\n");
                    liberar_cidades(cidades, n);
                    free(indices);
                    cidades = NULL;
                    indices = NULL;
                }

                if (carregar_dados(&cidades, &indices, &n) == 0) {
                    printf("Dados carregados com sucesso! (%d municipios)\n", n);
                } else {
                    printf("Falha ao carregar dados.\n");
                }
                break;
            }

            case 2: {
                if (cidades == NULL) {
                    printf("Carregue os dados primeiro (opcao 1).\n");
                    break;
                }

                int codigo_ibge;
                printf("Digite o codigo IBGE do municipio: ");
                if (scanf("%d", &codigo_ibge) != 1) {

```

```
int c;
while ((c = getchar()) != '\n' && c != EOF);
printf("Codigo invalido.\n");
break;
}
```

```
struct timespec t_ini, t_fim;
```

```
clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &t_ini);
int pos_sequencial = busca_linear(indices, n, codigo_ibge);
clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &t_fim);
double tempo_sequencial = (t_fim.tv_sec - t_ini.tv_sec) +
    (t_fim.tv_nsec - t_ini.tv_nsec) / 1e9;
```

```
clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &t_ini);
int pos_binaria = busca_binaria(indices, n, codigo_ibge);
clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &t_fim);
double tempo_binaria = (t_fim.tv_sec - t_ini.tv_sec) +
    (t_fim.tv_nsec - t_ini.tv_nsec) / 1e9;
```

```
if (pos_sequencial == -1) {
    printf("Municipio nao encontrado.\n");
} else {
    print_municipio(&idades[indices[pos_sequencial].linha]);
}
```

```
printf("Tempo Busca Sequencial: %.9f segundos (posicao %d)\n",
    tempo_sequencial, pos_sequencial);
printf("Tempo Busca Binaria:   %.9f segundos (posicao %d)\n",
    tempo_binaria, pos_binaria);
break;
}
```

```
case 3:
    printf("Encerrando...\n");
    break;
```

```
default:
    printf("Opcao invalida.\n");
}
```

```
} while (opcao != 3);
```

```
if (idades != NULL) {
    liberar_idades(idades, n);
    free(indices);
}
```

```

    return 0;
}

int carregar_dados(cidade **out_cidades, indice **out_indices, int *out_n) {
    FILE *file = fopen("municipios.csv", "r");
    if (file == NULL) {
        printf("Erro ao abrir o arquivo!\n");
        return -1;
    }

    char buffer[1024];
    cidade *cidades = (cidade*) malloc(QTD_CIDADES * sizeof(cidade));
    indice *indices = (indice*) malloc(QTD_CIDADES * sizeof(indice));
    if (cidades == NULL || indices == NULL) {
        printf("Erro de alocação de memória!\n");
        return -1;
    }
    fgets(buffer, 1024, file);

    int index_cidade = 0;
    while (fgets(buffer, 1024, file) != NULL && index_cidade < QTD_CIDADES) {
        int size = strlen(buffer);
        int start = 0, end = 0, contador_field = 0;

        for (int i = 0; i < size && contador_field <= 7; i++) {
            if (buffer[i] == ';') {
                end = i - 1;
                char *campo = substring_by_index(buffer, start, end);

                switch (contador_field) {
                    case 1:
                        cidades[index_cidade].ibge = atoi(campo);
                        free(campo);
                        break;
                    case 3:
                        cidades[index_cidade].uf = campo;
                        break;
                    case 4:
                        cidades[index_cidade].municipio = campo;
                        break;
                    case 5:
                        cidades[index_cidade].regiao = campo;
                        break;
                    case 6:
                        cidades[index_cidade].populacao = atol(campo);
                        free(campo);
                        break;
                }
            }
        }
        index_cidade++;
    }
}

```

```

        case 7:
            cidades[index_cidade].porte = campo;
            break;
        default:
            free(campo);
            break;
    }

    contador_field++;
    start = i + 1;
}

indices[index_cidade].ibge = cidades[index_cidade].ibge;
indices[index_cidade].linha = index_cidade;

index_cidade++;
}

fclose(file);

*out_cidades = cidades;
*out_indices = indices;
*out_n = index_cidade;
printf("%d\n", index_cidade);

return 0;
}

int busca_linear(indice *arr, int n, int required_ibge) {
    for (int i = 0; i < n; i++) {
        if (arr[i].ibge == required_ibge) {
            return i;
        }
    }
    return -1;
}

int busca_binaria(indice *arr, int n, int required_ibge) {
    int inicio = 0, fim = n - 1;

    while (inicio <= fim) {
        int meio = inicio + (fim - inicio) / 2;

        if (arr[meio].ibge == required_ibge) {
            return meio;
        } else if (arr[meio].ibge < required_ibge) {
            inicio = meio + 1;
        }
    }
}

```

```

    } else {
        fim = meio - 1;
    }
}

return -1;
}

void print_municipio(cidade *c) {
    printf("-----\n");
    printf("Municipio: %s\n", c->municipio);
    printf("UF: %s\n", c->uf);
    printf("Regiao: %s\n", c->regiao);
    printf("Populacao (2010): %ld\n", c->populacao);
    printf("Porte: %s\n", c->porte);
    printf("-----\n");
}

void liberar_cidades(cidade *arr, int n) {
    for (int i = 0; i < n; i++) {
        free(arr[i].municipio);
        free(arr[i].uf);
        free(arr[i].regiao);
        free(arr[i].porte);
    }
    free(arr);
}

char* substring_by_index(char *src, int start, int end) {
    int length = end - start + 1;
    if (length < 0) length = 0;

    char *new_str = (char *) malloc((length + 1) * sizeof(char));
    if (new_str == NULL) {
        return NULL;
    }

    if (length > 0) {
        strncpy(new_str, src + start, length);
    }
    new_str[length] = '\0';
    return new_str;
}

```